THE UPDATE ACTION

Updating a document follows a similar pattern, but we must wrap the fields to be updated in a `doc` object, as we learned in section 5.4.2. This example updates the movie with ID 200 (*Rush Hour*) with additional fields (`director` and `actors`).

Listing 5.47 Bulk updating the move *Rush Hour*

```
POST _bulk
{"update":{"_index":"movies","_id":"200"}}
{"doc": {"director":"Brett Ratner", "actors":["Jackie Chan","Chris Tucker"]}}
```

THE DELETE ACTION

Finally, let's use the `_bulk` API to delete a document. The format is slightly different, as shown next.

Listing 5.48 Bulk call with the `delete` action

```
POST _bulk
{"delete":{"_index":"movie_reviews","_id":"111"}}
```

As you can see, we don't need a second line for this operation. The query deletes the movie review with ID 111 from the `movie_reviews` index. Remember, you can lose your entire data set if you are not careful when issuing `delete` queries.

5.6.4 Bulk requests using cURL

We've been using Kibana to perform operations on documents using the `_bulk` API. We can also perform these actions using cURL. In fact, using cURL may be the preferred method if we have a larger chunk of records to deal with.

To use cURL, we need to create a JSON file containing all the data and pass that file to cURL with the flag `--data-binary`. The data used in listing 5.49 is available in the movie_bulk_data.json file available with the book's files; you can pass the file to cURL.

Listing 5.49 Using cURL to execute a bulk data operation

```
curl -H "Content-Type: application/x-ndjson"
  -XPOST localhost:9200/_bulk
  --data-binary "@movie_bulk_data.json"
```

The `localhost` is the address of our instance of Elasticsearch that is up and running on our local machine.

> **NOTE** Be sure to give the `--data-binary` flag the name of your file (without the extension) with a `@` prefix.

We now know how to work with bulk requests. However, sometimes we want to move documents from one index to another (for example, from `blockbuster_movies` to